GESTION DES LIVRAISONS



Glenn GUILLARD

22/10/2026

01 INTRODUCTION

Une entreprise de livraison doit préparer quotidiennement le chargement de ses camions. Elle dispose d'une liste de colis, chacun décrit par un identifiant, un poids, une valeur, une priorité et une destination, et cette liste peut atteindre 100 000 éléments. Chaque camion ayant une capacité limitée, tous les colis ne peuvent pas être transportés.

Le but est donc de développer un programme qui permet d'ajouter, d'afficher, de rechercher et de trier les colis, de calculer un chargement de valeur maximale, d'y intégrer la priorité, et de mesurer les performances lorsque le nombre de colis augmente.

Le langage choisi est Python, pour des raisons de simplification. Il n'y a pas de gestion de la mémoire. En effet, Python est plus verbeux et aussi plus simple en écriture. Cela ne change pas le résultat si le C avait été choisi.

02 STOCKAGE DES COLIS/RECHERCHE PAR IDENTIFIANT

Les colis sont stockés dans une liste, qui sert à l'affichage et aux tris, et dans un dictionnaire qui associe chaque identifiant à son colis, pour la recherche. Les deux contiennent les mêmes objets, sans copie.

Élément	Description
Problème	Retrouver rapidement un colis par son identifiant, parmi jusqu'à 100 000 colis.
Solution	Un dictionnaire Python (table de hachage).
Temps	O(1) en moyenne, O(n) dans le pire cas.
Espace	O(n).
Alternative	Parcourir la liste, en O(n).
Justification	La recherche est fréquente et doit rester rapide. Pour 1 000 recherches sur 100 000 colis : environ 1 300 ms avec la liste, contre 0,5 ms avec le dictionnaire.

Python calcule le hash de l'identifiant, qui indique directement la case où se trouve le colis, on y accède sans parcourir les autres.
Si deux identifiants tombent sur la même case (une collision), Python essaie d'autres cases. Le tableau est agrandi dès qu'il est rempli aux deux tiers : il reste donc peu rempli, et on trouve un colis en quelques essais, quelle que soit la taille de la collection.

03 TRI PAR POIDS ET PAR VALEUR

Les colis peuvent être affichés par poids croissant ou par valeur décroissante. Le tri se fait sur une copie de la liste, pour ne pas modifier l'ordre d'origine.

Élément	Description
Problème	Trier jusqu'à 100 000 colis par poids ou par valeur.
Solution	Un tri fusion, codé à la main.
Temps	O(n log n), dans tous les cas.
Espace	O(n), pour un tableau auxiliaire.
Alternative	Le tri par insertion, en $O(n^2)$, ou sorted(), le tri natif de Python (Timsort).
Justification	Le tri par insertion devient beaucoup trop lent : environ 2,9 s pour seulement 10 000 colis. Le tri fusion trie 100 000 colis en environ 280 ms, et sa complexité est garantie quel que soit l'ordre des données.

On découpe la liste en petits blocs, que l'on fusionne deux par deux : des blocs de 1 élément, puis de 2, de 4, de 8, jusqu'à la liste entière. Pour fusionner deux blocs déjà triés, on compare leurs premiers éléments et on prend le plus petit, ce qui coûte un temps proportionnel à leur taille. Il y a log n étapes de fusion, chacune en O(n), d'où le O(n log n).

04 TRI PAR PRIORITÉ

Les colis peuvent être affichés par priorité décroissante. La priorité ne prend que 5 valeurs possibles, de 1 à 5.

Élément	Description
Problème	Trier jusqu'à 100 000 colis par priorité, qui ne prend que 5 valeurs.
Solution	Un tri par comptage (avec des « seaux »), codé à la main.
Temps	O(n + k), avec k = 5 valeurs possibles, soit O(n).
Espace	O(n + k).
Alternative	Réutiliser le tri fusion, en O(n log n).
Justification	Le tri par comptage est linéaire, donc plus rapide : environ 27 ms pour 100 000 colis, contre 280 ms

On crée un « seau » par niveau de priorité, soit 5 seaux. On parcourt les colis une seule fois et on range chacun dans le seau de sa priorité. Il suffit ensuite de mettre les seaux bout à bout, de la priorité 5 à la priorité 1. Aucun colis n'est comparé à un autre : c'est ce qui permet de descendre sous le O(n log n).

05 CHARGEMENT DU CAMION

Le camion a une capacité maximale W. Il faut choisir les colis à charger pour que le poids total ne dépasse pas W, tout en obtenant la plus grande valeur totale possible. Chaque colis est soit pris, soit laissé : c'est le problème du sac à dos.

Élément	Description
Problème	Trouver le chargement de valeur maximale qui respecte la capacité du camion.
Solution	La programmation dynamique.
Temps	O(n × W), avec n le nombre de colis et W la capacité.
Espace	O(n × W), pour mémoriser les choix et retrouver les colis chargés.
Alternative	La force brute, en $O(2^n)$, ou le glouton, en O(n log n).
Justification	C'est la seule approche à la fois optimale et utilisable avec beaucoup de colis.

06 RÈGLE DE PRIORITÉ

Pour favoriser les colis prioritaires, chaque colis reçoit un score, que l'algorithme de chargement maximise à la place de la valeur :

$$\text{score} = \text{valeur} \times (10 + \text{alpha} \times (\text{priorité} - 1))$$

- Le poids reste la contrainte du camion.
- La valeur reste la base du score : un colis prioritaire sans valeur n'est pas favorisé.
- La priorité multiplie la valeur : avec alpha = 5, un colis de priorité 5 compte 3 fois sa valeur. Avec alpha = 0, la priorité est ignorée.

Exemple avec les colis A à D et une capacité de 12 kg :

- sans priorité, le programme charge B et C, pour une valeur de 43 ;
- avec la priorité, il charge A et D, pour une valeur de 34, mais avec des priorités de 5 et 3 au lieu de 1 et 4.

07 PERFORMANCES

Temps mesurés en millisecondes, sur des colis générés aléatoirement :

n	Recherche liste	Recherche dict	Tri fusion	Tri comptage	Chargement (200 kg)
100	1,6	0,1	0,13	0,03	2,4
1 000	12,8	0,1	1,8	0,27	22,8
10 000	123,7	0,16	21,7	2,6	230
100 000	1 296	0,5	283	27	2 358

Capture de l'option 9 du menu (mesures de performances) :

Mesures de performances

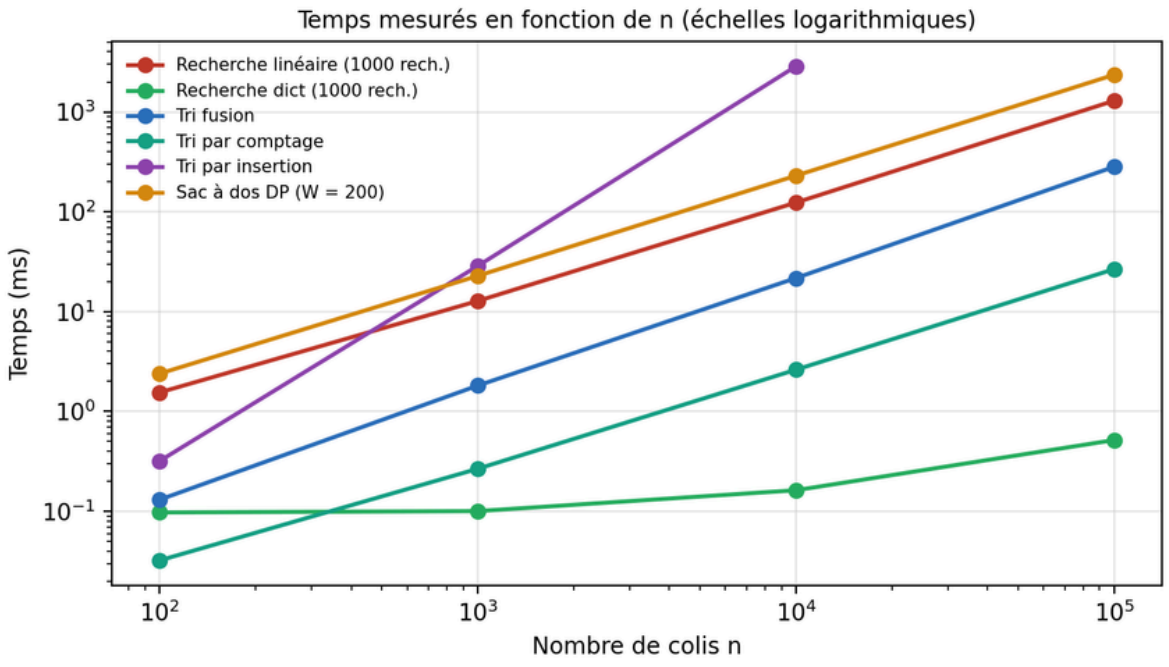
```
$ python3 src/main.py

[menu] Choix : 9

--- Mesures de performances (temps en ms) ---
Recherches : 1000 recherches d'identifiants existants.
Sac à dos  : capacité 200 kg. Tri par insertion ignoré au-delà de 10000 colis.
sorted() = tri natif de Python (Timsort), donné pour comparaison.

       n |    rech. lin. |    rech. dict |    tri fusion |      sorted() |  tri comptage | tri insertion |  sac a dos DP |
---------+---------------+---------------+---------------+---------------+---------------+---------------+---------------+
     100 |         1.506 |         0.103 |         0.131 |         0.019 |         0.033 |         0.263 |         2.397 |
    1000 |        12.746 |         0.092 |         1.658 |         0.154 |         0.259 |        26.032 |        23.145 |
   10000 |       125.122 |         0.201 |        21.575 |         1.519 |         2.456 |      2611.633 |       231.936 |
  100000 |      1306.300 |         0.470 |       279.425 |        15.573 |        26.218 |             - |      2355.125 |
```

Temps mesurés en fonction du nombre de colis (échelles logarithmiques) :



08 Annexe : captures d'exécution

Menu et affichage des colis (option 1) :

Affichage des colis

```
$ python3 src/main.py

===== GESTION DES LIVRAISONS =====
1. Afficher les colis
2. Ajouter un colis
3. Rechercher un colis
4. Organiser les colis
5. Préparer un camion
6. Afficher le chargement
7. Afficher les statistiques
8. Générer des colis aléatoires
9. Mesurer les performances
0. Quitter
Choix : 1

--- Liste des colis (5) ---
ID       Poids    Valeur   Priorité  Destination
-------- -------- -------- --------- ------------------
101      8        20       2         Paris
102      3        10       5         Lille
103      12       40       1         Lyon
104      5        25       4         Paris
105      2        15       3         Lille
```

Ajout puis recherche d'un colis (options 2 et 3) :

Ajout et recherche d'un colis

```
$ python3 src/main.py

[menu] Choix : 2

--- Ajouter un colis ---
Identifiant : 106
Poids (kg) : 7
Valeur : 30
Priorité (1 = faible ... 5 = forte) : 5
Destination : Rennes
Colis 106 ajouté.

[menu] Choix : 3

--- Rechercher un colis ---
Identifiant : 106
ID       Poids    Valeur   Priorité  Destination
-------- -------- -------- --------- ------------------
106      7        30       5         Rennes

[menu] Choix : 3

--- Rechercher un colis ---
Identifiant : 999
Aucun colis avec l'identifiant 999.
```

Tri par poids (tri fusion) puis par priorité (tri par comptage), option 4 :

Tris par poids et par priorité

```
$ python3 src/main.py

[menu] Choix : 4

--- Organiser les colis ---
1. Par poids (croissant)       - tri fusion
2. Par valeur (décroissante)   - tri fusion
3. Par priorité (décroissante) - tri par comptage
Choix : 1
ID       Poids    Valeur   Priorité  Destination
-------- -------- -------- --------- ------------------
105      2        15       3         Lille
102      3        10       5         Lille
104      5        25       4         Paris
101      8        20       2         Paris
103      12       40       1         Lyon
Tri de 5 colis en 0.015 ms.

[menu] Choix : 4

--- Organiser les colis ---
1. Par poids (croissant)       - tri fusion
2. Par valeur (décroissante)   - tri fusion
3. Par priorité (décroissante) - tri par comptage
Choix : 3
ID       Poids    Valeur   Priorité  Destination
-------- -------- -------- --------- ------------------
102      3        10       5         Lille
104      5        25       4         Paris
105      2        15       3         Lille
101      8        20       2         Paris
103      12       40       1         Lyon
Tri de 5 colis en 0.005 ms.
```

Chargement d'un camion de 17 kg en maximisant la valeur (option 5, critère 1). La programmation dynamique obtient une valeur de 65, contre 50 pour le glouton, et la force brute confirme que c'est l'optimum :

Chargement du camion (valeur)

```
$ python3 src/main.py

[menu] Choix : 5

--- Préparer un camion ---
Capacité du camion (kg) : 17
1. Maximiser la valeur
2. Maximiser la valeur en favorisant la priorité
Critère : 1

Chargement optimal (programmation dynamique, 0.034 ms) :
ID       Poids    Valeur   Priorité  Destination
-------- -------- -------- --------- --------------------
103      12       40       1         Lyon
104      5        25       4         Paris

Colis chargés  : 2
Poids total    : 17 / 17 kg (100.0 %)
Valeur totale  : 65
Score optimisé : 65

Comparaison glouton (ratio gain/poids) : score 50 (valeur 50, 10 kg) -> moins bon de 15
Vérification force brute : score 65 -> optimal confirmé
```

Même camion en favorisant la priorité (critère 2, alpha = 5). Les colis 101, 102 et 104 (priorités 2, 5 et 4) remplacent le colis 103 (priorité 1) : la valeur baisse à 55, mais les colis urgents sont livrés :

Chargement du camion (priorité)

```
$ python3 src/main.py

[menu] Choix : 5

--- Préparer un camion ---
Capacité du camion (kg) : 17
1. Maximiser la valeur
2. Maximiser la valeur en favorisant la priorité
Critère : 2
Poids de la priorité alpha (0 = ignorée, 5 = conseillé) : 5

Chargement optimal (programmation dynamique, 0.029 ms) :
ID       Poids    Valeur   Priorité  Destination
-------- -------- -------- --------- --------------------
101      8        20       2         Paris
102      3        10       5         Lille
104      5        25       4         Paris

Colis chargés  : 3
Poids total    : 16 / 17 kg (94.1 %)
Valeur totale  : 55
Score optimisé : 1225

Comparaison glouton (ratio gain/poids) : score 1225 (valeur 50, 10 kg) -> identique ici
Vérification force brute : score 1225 -> optimal confirmé
```